EE/EC4202 Computer Architecture and Organization
Programming Assignments: Intel x86 Assembly
Department of Electrical and Information Engineering, University of Ruhuna

General Instructions

Environment: 32-bit NASM on Linux (`nasm -f elf32`, `ld -m elf_i386`), using `int 0x80` system calls.

Rules:
Comment every logical block. Use meaningful labels.
Do not use C library functions (no `printf`, no `scanf`) only raw system calls.
Individual work only. Plagiarism will be penalised.

Assignment 1: Registers, Data Movement and Arithmetic/Logic

Topics: registers, addressing modes, `mov`, `add`, `sub`, `mul`, `div`, `and`, `or`, `xor`, `shl`, `shr`, flags, number systems

Task 1 (Arithmetic):
Declare two 32-bit unsigned integers in the `.data` section. Compute and store in memory:
- their sum, difference, and product (use `mul`, and keep the full 64-bit result in `EDX:EAX`)
- their quotient and remainder (use `div`)

Task 2 (Bit manipulation).
Given a 32-bit value in memory, write code that:
- counts the number of `1` bits (using `shr` and the carry flag)
- checks whether the value is even or odd using `and`
- swaps its upper and lower 16 bits (using `shl`, `shr`, `or`)
- checks whether it is a power of two (using `x & (x-1)`)

Task 3 (Addressing modes).
Write a short program that demonstrates each of these addressing modes with a one-line comment explaining the effective address: immediate, register, direct, register indirect, base + displacement, and base + index * scale.

Deliverable: Since you have not yet learned output routines, verify results by using `gdb` (`info registers`, `x/4dw &var`) and include screenshots. Alternatively, return the result as the exit code (`echo $?`) for values under 256.

Discussion questions:

1. What are the differences between the flags set by `add` and `sub`? Which flag signals unsigned overflow and which signals signed overflow?
2. Why does `mul` place its result in `EDX:EAX`? What happens if the result does not fit in 32 bits and you only read `EAX`?
3. Explain the difference between `mov eax, var` and `mov eax, [var]` in NASM.

Assignment 2: Control Flow, Loops and Arrays

Topics: `cmp`, `jmp`, conditional jumps (`je`, `jne`, `jl`, `jg`, `ja`, `jb`...), `loop`, array indexing, signed vs unsigned comparison

Task 1 (Array statistics).

Given an array of at least 10 signed 32-bit integers in `.data`, compute:
- the sum and the average (integer division)
- the maximum and minimum values
- the number of positive, negative, and zero elements

Task 2 (Sorting).

Implement **bubble sort** to sort the array in ascending order, in place. Then modify the code so that a single flag (memory byte) selects ascending or descending order.

Task 3 (Search).

Implement **binary search** on the sorted array for a target value. Store the index in a variable, or `-1` if it is not found.

Task 4 (Number sequences).

Compute the first N Fibonacci numbers into an array (N >= 15) and find the largest Fibonacci number that fits in an unsigned 32-bit register (detect overflow using the carry flag).

Deliverable: Use `gdb` to show the final state of the arrays and variables. Test with at least three different arrays, including one with all-equal elements and one already sorted.

Discussion questions :

1. What is the difference between `jg`/`jl` and `ja`/`jb`? Give an example where they produce different results.
2. Count the number of instructions executed in the inner loop of your bubble sort. Estimate the total for N = 10.
3. Why does `loop` have a limited range, and how would you write an equivalent loop without it?

Assignment 3: The Stack, Procedures and Recursion

Topics: `push`, `pop`, `call`, `ret`, stack frames (`EBP`/`ESP`), calling convention (cdecl), local variables, recursion

Task 1 (Calling convention).

Write the following as separate procedures that receive arguments on the stack (cdecl: arguments pushed right to left, caller cleans up, result in `EAX`) and use a proper stack frame (`push ebp` / `mov ebp, esp` ... `leave` / `ret`):
- `int max2(int a, int b)`
- `int power(int base, int exp)` (iterative)
- `int array_sum(int *arr, int n)`

Task 2 (Recursion).

Implement recursively:
- `factorial(n)`
- `gcd(a, b)` using Euclid's algorithm
- `fib(n)`

Draw the stack contents (as a diagram in your report) at the deepest point of the call `factorial(4)`.

Task 3 (Register preservation).

Write a procedure that uses `EBX`, `ESI`, and `EDI` internally and correctly preserves them for the caller. Demonstrate the bug that appears when you fail to preserve `EBX`, and show how you found it with `gdb`.

Task 4 (Local variables).

Write a procedure that allocates two local variables on the stack using `sub esp, 8`, uses them, and releases them correctly.

Deliverable: A main program that calls every procedure with test values and leaves each result in memory or as an exit code, verified in `gdb`.

Discussion questions:

1. What exactly happens to `ESP` and `EIP` during `call` and `ret`?
2. What happens if a procedure pops fewer or more values than it pushed? Demonstrate it.
3. Estimate the stack memory needed for `factorial(1000)` if each frame is 12 bytes. What would happen on overflow?

Assignment 4: Strings, System Calls and I/O

Topics: `sys_read`, `sys_write`, `sys_exit`, ASCII, string instructions (`movsb`, `lodsb`, `stosb`, `scasb`, `cmpsb`, `rep`), integer <-> string conversion

Task 1 (Output routines).

Write reusable procedures (you may reuse your Assignment 3 calling convention):
- `print_string(ptr, len)` using `sys_write`
- `print_uint(n)` converting an unsigned integer to decimal ASCII (repeated `div` by 10, then reverse the digits) and printing it
- `print_hex(n)` printing a value as 8 hex digits

Task 2 (Input).

Write `read_int()` which reads a line from the keyboard using `sys_read`, validates the characters, and converts the decimal string to an integer in `EAX`. Reject invalid input with an error message.

Task 3 (String processing).

Read a line of text from the user and:
- print its length
- print it reversed
- print it converted to uppercase (using only `and`/`sub` on ASCII values)
- report whether it is a **palindrome**, ignoring case
- count the vowels

Use at least one string instruction with the `rep` prefix (for example `rep movsb` to copy the string).

Task 4 (Mini application).

Combine the routines into a menu-driven program that repeats until the user quits:

1. Add two numbers
2. Factorial of a number
3. Palindrome check
4. Exit

Deliverable: Screenshots of the program running with normal input and edge cases (empty input, very long input, non-numeric input, `0`, and the largest 32-bit value).

Discussion questions:

1. Explain the role of the direction flag (`DF`) with string instructions. What happens if it is set incorrectly?
2. How does your program behave if the user types more characters than your buffer holds? Describe the security risk this represents.
3. List the system-call number, and the register used for each argument, for `sys_read` and `sys_write`. Where in the Linux kernel is this table defined?

Submission:

1. Source file(s): Submit your assembly source file(s) named **<name>_<RegNo>.asm**

2. Report: Submit a short report (maximum 2 pages, PDF) containing a description of your approach screenshots of your test runs answers to the discussion questions. Clearly state your own assumptions and answers.

3. Zip File : Place each assignment's .asm files and report in a separate folder named assignment_n (n = 1 to 4). Compress all four folders into a **single zip** file and upload it named **<name>_<RegNo>.**